

Rappel : addition et soustraction écrite en base 10

Additions

① $347 + 586 = 933$

$$\begin{array}{r}
 1\ 1 \quad <- \text{retenues} \\
 3\ 4\ 7 \\
 +\ 5\ 8\ 6 \\
 \hline
 9\ 3\ 3
 \end{array}$$

Unités : $7+6=13 \rightarrow$ pose 3, retiens 1. Dizaines : $4+8+1=13 \rightarrow$ pose 3, retiens 1. Centaines : $3+5+1=9$.

② $4728 + 3695 = 8423$

$$\begin{array}{r}
 1\ 1\ 1 \quad <- \text{retenues} \\
 4\ 7\ 2\ 8 \\
 +\ 3\ 6\ 9\ 5 \\
 \hline
 8\ 4\ 2\ 3
 \end{array}$$

$u : 8+5=13 \rightarrow 3$, ret 1. $d : 2+9+1=12 \rightarrow 2$, ret 1. $c : 7+6+1=14 \rightarrow 4$, ret 1. $m : 4+3+1=8$.

③ $2856 + 1744 = 4600$

$$\begin{array}{r}
 1\ 1\ 1 \quad <- \text{retenues} \\
 2\ 8\ 5\ 6 \\
 +\ 1\ 7\ 4\ 4 \\
 \hline
 4\ 6\ 0\ 0
 \end{array}$$

$u : 6+4=10 \rightarrow 0$, ret 1. $d : 5+4+1=10 \rightarrow 0$, ret 1. $c : 8+7+1=16 \rightarrow 6$, ret 1. $m : 2+1+1=4$.

④ $68 + 57 = 125$

$$\begin{array}{r}
 1\ 1 \quad <- \text{retenues} \\
 6\ 8 \\
 +\ 5\ 7 \\
 \hline
 1\ 2\ 5
 \end{array}$$

$u : 8+7=15 \rightarrow 5$, ret 1. $d : 6+5+1=12 \rightarrow 2$, ret 1 $\rightarrow$ cette retenue crée le chiffre des centaines (1).

⑤ $9074 + 926 = 10000$

$$\begin{array}{r}
 1\ 1\ 1\ 1 \quad <- \text{retenues} \\
 9\ 0\ 7\ 4 \\
 +\ 9\ 2\ 6 \\
 \hline
 1\ 0\ 0\ 0\ 0
 \end{array}$$

$u : 4+6=10 \rightarrow 0$, ret 1. $d : 7+2+1=10 \rightarrow 0$, ret 1. $c : 0+9+1=10 \rightarrow 0$, ret 1. $m : 9+0+1=10 \rightarrow 0$, ret 1 $\rightarrow$ nouvelle colonne des dizaines de mille (1). Cascade complète de retenues.

Soustractions

⑥ $802 - 547 = 255$

$$\begin{array}{r}
 8\ 0\ 2 \\
 -\ 5\ 4\ 7
 \end{array}$$

2 5 5

u : 2-7 impossible → emprunt : 12-7=5. La dizaine est 0 : l'emprunt se propage sur la centaine (8 → 7) et la dizaine devient 9. d : 9-4=5. c : 7-5=2.

⑦ **1000 - 348 = 652**

1 0 0 0
- 3 4 8

6 5 2

u : 0-8 impossible → emprunt en cascade à travers les zéros. Le 1 des milliers prête : u devient 10 → 10-8=2, d devient 9 → 9-4=5, c devient 9 → 9-3=6, m devient 0.

⑧ **5231 - 2867 = 2364**

5 2 3 1
- 2 8 6 7

2 3 6 4

u : 1-7 → emprunt : 11-7=4 (d : 3 → 2). d : 2-6 → emprunt : 12-6=6 (c : 2 → 1). c : 1-8 → emprunt : 11-8=3 (m : 5 → 4). m : 4-2=2.

⑨ **640 - 285 = 355**

6 4 0
- 2 8 5

3 5 5

u : 0-5 → emprunt : 10-5=5 (d : 4 → 3). d : 3-8 → emprunt : 13-8=5 (c : 6 → 5). c : 5-2=3.

⑩ **7003 - 1258 = 5745**

7 0 0 3
- 1 2 5 8

5 7 4 5

u : 3-8 → emprunt en cascade à travers les deux zéros. Le 7 des milliers prête : u devient 13 → 13-8=5, d devient 9 → 9-5=4, c devient 9 → 9-2=7, m : 6-1=5.

Astuce de vérification

Pour une soustraction $a - b = r$, on peut vérifier en additionnant : $r + b$ doit redonner a .

Exemple ⑧ : $2364 + 2867 = 5231$. Pour une addition, on peut recompter dans l'autre sens ($b + a$).

Exercice 1 : réalise les opérations suivantes

Colonne 1 — additions

① $1001 + 0110 = 1111$ ($9 + 6 = 15$)

```
  1 0 0 1
+ 0 1 1 0
-----
  1 1 1 1
```

Aucune retenue : $1+0$, $0+1$, $0+1$, $1+0$.

② $10101 + 1011 = 10000$ ($21 + 11 = 32$)

```
   1 1 1 1 1   <- retenues/emprunts
  1 0 1 0 1
+   1 0 1 1
-----
  1 0 0 0 0 0
```

À chaque colonne $1+1$ (ou $1+1+retenue$) → on pose 0 et on retient 1, jusqu'au débordement final.

③ $100 + 111 = 1011$ ($4 + 7 = 11$)

```
      1   <- retenues/emprunts
    1 0 0
+   1 1 1
-----
  1 0 1 1
```

pos2 : $1+1=10$ → pose 0, retiens 1 → nouveau bit de poids fort.

④ $1010 + 0101 = 1111$ ($10 + 5 = 15$)

```
  1 0 1 0
+ 0 1 0 1
-----
  1 1 1 1
```

Aucune retenue : les 1 et les 0 alternent parfaitement.

⑤ $11000 + 00111 = 11111$ ($24 + 7 = 31$)

```
  1 1 0 0 0
+ 0 0 1 1 1
-----
  1 1 1 1 1
```

Aucune retenue : chaque colonne fait $1+0$ ou $0+1$.

Colonne 2 — additions

⑥ $100 + 111 = 1011$ ($4 + 7 = 11$)

```
      1   <- retenues/emprunts
    1 0 0
+   1 1 1
-----
  1 0 1 1
```

Identique au ③.

⑦ $1001 + 1010 = 10011$ ($9 + 10 = 19$)

```
      1   <- retenues/emprunts
```

```

      1 0 0 1
+     1 0 1 0
-----
      1 0 0 1 1

```

pos3 : 1+1=10 → pose 0, retiens 1.

⑧ **10101 + 11001 = 101110 (21 + 25 = 46)**

```

      1 0 0 0 1  <- retenues/emprunts
      1 0 1 0 1
+     1 1 0 0 1
-----
      1 0 1 1 1 0

```

pos0 : 1+1=10 → retenue ; pos4 : 1+1=10 → débordement.

⑨ **1111 + 1001 = 11000 (15 + 9 = 24)**

```

      1 1 1 1  <- retenues/emprunts
      1 1 1 1
+     1 0 0 1
-----
      1 1 0 0 0

```

Cascade de retenues : pos0..2 donnent 10 (pose 0), pos3 : 1+1+1=11 (pose 1, retiens 1).

⑩ **10101 + 1010 = 11111 (21 + 10 = 31)**

```

      1 0 1 0 1
+     1 0 1 0
-----
      1 1 1 1 1

```

Aucune retenue.

Colonne 3 — soustractions

⑪ **1001 - 0110 = 11 (9 - 6 = 3)**

```

      1 0 0 1
-     0 1 1 0
-----
      0 0 1 1

```

pos1 : 0-1 → emprunt ; l'emprunt se propage sur pos2.

⑫ **1100 - 0100 = 1000 (12 - 4 = 8)**

```

      1 1 0 0
-     0 1 0 0
-----
      1 0 0 0

```

Aucun emprunt : pos2 : 1-1=0.

⑬ **1010 - 0101 = 101 (10 - 5 = 5)**

```

      1 0 1 0
-     0 1 0 1
-----
      0 1 0 1

```

Deux emprunts : pos0 (0-1) et pos2 (0-1).

⑭ **111 - 011 = 100 (7 - 3 = 4)**

```

      1 1 1
-     0 1 1

```

```

-----
1 0 0

```

Aucun emprunt : $pos0$ et $pos1$: $1-1=0$; $pos2$: $1-0=1$.

⑮ $1010 - 1000 = 10$ ($10 - 8 = 2$)

```

  1 0 1 0
- 1 0 0 0
-----
  0 0 1 0

```

Aucun emprunt : seule $pos1$ vaut 1.

Colonne 4 — soustractions

⑯ $1001 - 1010 = -1$ ($9 - 10 = -1$)

$1010 > 1001$: résultat négatif. On calcule $1010 - 1001$, puis on met le signe $-$.

```

  1 0 1 0
- 1 0 0 1
-----
  0 0 0 1

```

Résultat : -1 .

⑰ $10101 - 11001 = -100$ ($21 - 25 = -4$)

$11001 > 10101$: résultat négatif. On calcule $11001 - 10101$, puis on met le signe $-$.

```

  1 1 0 0 1
- 1 0 1 0 1
-----
  0 0 1 0 0

```

Résultat : -100 .

⑱ $1111 - 1001 = 110$ ($15 - 9 = 6$)

```

  1 1 1 1
- 1 0 0 1
-----
  0 1 1 0

```

Aucun emprunt.

⑲ $1100 - 0111 = 101$ ($12 - 7 = 5$)

```

  1 1 0 0
- 0 1 1 1
-----
  0 1 0 1

```

Chaîne d'emprunts sur $pos0$, $pos1$ et $pos2$.

⑳ $10101 - 01110 = 111$ ($21 - 14 = 7$)

```

  1 0 1 0 1
- 0 1 1 1 0
-----
  0 0 1 1 1

```

Emprunts successifs de $pos1$ à $pos3$.

Remarque sur les résultats négatifs

Les opérations ⑯ et ⑰ donnent un nombre négatif car le nombre soustrait est le plus grand. Ici on l'écrit avec un signe $-$ à la main. Sur un support à taille fixe (par ex. 5 bits), ces valeurs se coderaient plutôt en complément à 2.

Exercice 2 : donne l'équivalent en base 10 des nombres suivants

A. Bases

$(10101)_2$

Bit : 1 0 1 0 1

Poids : 16 8 4 2 1

Somme : $16 + 4 + 1 = 21$

Bits à 1 en positions 2^4 , 2^2 et 2^0 .

$(11111)_2$

Bit : 1 1 1 1 1

Poids : 16 8 4 2 1

Somme : $16 + 8 + 4 + 2 + 1 = 31$

Tous les bits à 1 : c'est le plus grand nombre sur 5 bits, soit $2^5 - 1 = 31$.

$(100000)_2$

Bit : 1 0 0 0 0 0

Poids : 32 16 8 4 2 1

Somme : $32 = 32$

Un seul 1 en tête : c'est exactement $2^5 = 32$.

$(101010)_2$

Bit : 1 0 1 0 1 0

Poids : 32 16 8 4 2 1

Somme : $32 + 8 + 2 = 42$

Bits à 1 en positions 2^5 , 2^3 et 2^1 .

$(111000)_2$

Bit : 1 1 1 0 0 0

Poids : 32 16 8 4 2 1

Somme : $32 + 16 + 8 = 56$

Bits à 1 en positions 2^5 , 2^4 et 2^3 .

$(0)_2$

Bit : 0

Poids : 1

Somme : $0 = (0)_{10}$

Aucun bit à 1 : la somme est vide, donc la valeur est 0. Cas limite de base.

$(00101)_2$

Bit : 0 0 1 0 1

Poids : 16 8 4 2 1

Somme : $4 + 1 = (5)_{10}$

Les deux 0 de tête ne changent RIEN : $(00101)_2 = (101)_2 = 5$. Utile car les tailles fixes (octet) obligent à écrire ces zéros non significatifs.

B. Puissances de 2 et 1 isolés

③ (1000)₂

Bit : 1 0 0 0

Poids : 8 4 2 1

Somme : 8 = (8)₁₀

Un 1 suivi de 3 zéros = $2^3 = 8$. Réflexe : « 1 suivi de n zéros = 2^n ».

④ (10000)₂

Bit : 1 0 0 0 0

Poids : 16 8 4 2 1

Somme : 16 = (16)₁₀

Un 1 suivi de 4 zéros = $2^4 = 16$.

⑤ (10001)₂

Bit : 1 0 0 0 1

Poids : 16 8 4 2 1

Somme : 16 + 1 = (17)₁₀

Deux 1 isolés encadrant des zéros : 16 + 1 = 17.

⑥ (1001)₂

Bit : 1 0 0 1

Poids : 8 4 2 1

Somme : 8 + 1 = (9)₁₀

1 aux extrémités, 0 au milieu : 8 + 1 = 9.

C. Largeurs extrêmes (1 bit ... octet complet)

⑦ (1)₂

Bit : 1

Poids : 1

Somme : 1 = (1)₁₀

Le plus court possible : 1.

⑧ (11)₂

Bit : 1 1

Poids : 2 1

Somme : 2 + 1 = (3)₁₀

2 + 1 = 3.

⑨ (11001010)₂

Bit : 1 1 0 0 1 0 1 0

Poids : 128 64 32 16 8 4 2 1

Somme : 128 + 64 + 8 + 2 = (202)₁₀

Octet complet (8 bits) : 128 + 64 + 8 + 2 = 202. Lien hexadécimal : en groupant par 4 bits, 1100 = C et 1010 = A, donc (11001010)₂ = (CA)₁₆ = 202. C'est le même regroupement que pour les codes couleur.

Exercice 3 : donne l'équivalent en base 2 des nombres suivants

1. Bases

• $(5)_{10}$

$$5 \div 2 = 2 \quad \text{reste } 1$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(101)_2$

• $(8)_{10} = 2^3$

$$8 \div 2 = 4 \quad \text{reste } 0$$

$$4 \div 2 = 2 \quad \text{reste } 0$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(1000)_2$

• $(10)_{10}$

$$10 \div 2 = 5 \quad \text{reste } 0$$

$$5 \div 2 = 2 \quad \text{reste } 1$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(1010)_2$

• $(15)_{10} = 2^4 - 1$ (tout à 1)

$$15 \div 2 = 7 \quad \text{reste } 1$$

$$7 \div 2 = 3 \quad \text{reste } 1$$

$$3 \div 2 = 1 \quad \text{reste } 1$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(1111)_2$

• $(32)_{10} = 2^5$

$$32 \div 2 = 16 \quad \text{reste } 0$$

$$16 \div 2 = 8 \quad \text{reste } 0$$

$$8 \div 2 = 4 \quad \text{reste } 0$$

$$4 \div 2 = 2 \quad \text{reste } 0$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(100000)_2$

• $(255)_{10} = 2^8 - 1$ (octet plein)

$$255 \div 2 = 127 \quad \text{reste } 1$$

$$127 \div 2 = 63 \quad \text{reste } 1$$

$$63 \div 2 = 31 \quad \text{reste } 1$$

$$31 \div 2 = 15 \quad \text{reste } 1$$

$$15 \div 2 = 7 \quad \text{reste } 1$$

$$7 \div 2 = 3 \quad \text{reste } 1$$

$$3 \div 2 = 1 \quad \text{reste } 1$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

Résultat (restes lus de bas en haut) : $(11111111)_2$

• $(256)_{10} = 2^8$

$$256 \div 2 = 128 \quad \text{reste } 0$$

$$128 \div 2 = 64 \quad \text{reste } 0$$

64	÷	2	=	32	reste 0
32	÷	2	=	16	reste 0
16	÷	2	=	8	reste 0
8	÷	2	=	4	reste 0
4	÷	2	=	2	reste 0
2	÷	2	=	1	reste 0
1	÷	2	=	0	reste 1

Résultat (restes lus de bas en haut) : **(100000000)₂**

255 → 256 : passage de 11111111 (octet plein) à 100000000 (débordement sur un 9e bit).

2. Compléments (entiers) : cas limites, repères et nombre « quelconque »

• **(0)₁₀ (cas limite)**

0	÷	2	=	0	reste 0
---	---	---	---	---	---------

Résultat (restes lus de bas en haut) : **(0)₂**

• **(1)₁₀**

1	÷	2	=	0	reste 1
---	---	---	---	---	---------

Résultat (restes lus de bas en haut) : **(1)₂**

• **(7)₁₀ = 2³ - 1 (à comparer à 8 = 1000)**

7	÷	2	=	3	reste 1
---	---	---	---	---	---------

3	÷	2	=	1	reste 1
---	---	---	---	---	---------

1	÷	2	=	0	reste 1
---	---	---	---	---	---------

Résultat (restes lus de bas en haut) : **(111)₂**

• **(128)₁₀ = 2⁷**

128	÷	2	=	64	reste 0
-----	---	---	---	----	---------

64	÷	2	=	32	reste 0
----	---	---	---	----	---------

32	÷	2	=	16	reste 0
----	---	---	---	----	---------

16	÷	2	=	8	reste 0
----	---	---	---	---	---------

8	÷	2	=	4	reste 0
---	---	---	---	---	---------

4	÷	2	=	2	reste 0
---	---	---	---	---	---------

2	÷	2	=	1	reste 0
---	---	---	---	---	---------

1	÷	2	=	0	reste 1
---	---	---	---	---	---------

Résultat (restes lus de bas en haut) : **(10000000)₂**

• **(100)₁₀ (sans motif régulier)**

100	÷	2	=	50	reste 0
-----	---	---	---	----	---------

50	÷	2	=	25	reste 0
----	---	---	---	----	---------

25	÷	2	=	12	reste 1
----	---	---	---	----	---------

12	÷	2	=	6	reste 0
----	---	---	---	---	---------

6	÷	2	=	3	reste 0
---	---	---	---	---	---------

3	÷	2	=	1	reste 1
---	---	---	---	---	---------

1	÷	2	=	0	reste 1
---	---	---	---	---	---------

Résultat (restes lus de bas en haut) : **(1100100)₂**

• **(2025)₁₀ (grand nombre)**

2025	÷	2	=	1012	reste 1
------	---	---	---	------	---------

1012	÷	2	=	506	reste 0
------	---	---	---	-----	---------

506	÷	2	=	253	reste 0
-----	---	---	---	-----	---------

253	÷	2	=	126	reste 1
-----	---	---	---	-----	---------

126	÷	2	=	63	reste 0
-----	---	---	---	----	---------

63	÷	2	=	31	reste 1
----	---	---	---	----	---------

31	÷	2	=	15	reste 1
----	---	---	---	----	---------

15	÷	2	=	7	reste 1
7	÷	2	=	3	reste 1
3	÷	2	=	1	reste 1
1	÷	2	=	0	reste 1

Résultat (restes lus de bas en haut) : **(11111101001)₂**

Récapitulatif

(5) ₁₀	=	101
(8) ₁₀	=	1000
(10) ₁₀	=	1010
(15) ₁₀	=	1111
(32) ₁₀	=	100000
(255) ₁₀	=	11111111
(256) ₁₀	=	100000000
(0) ₁₀	=	0
(1) ₁₀	=	1
(7) ₁₀	=	111
(128) ₁₀	=	10000000
(100) ₁₀	=	1100100
(2025) ₁₀	=	11111101001

Exercice 4 : donne le codage en complément à 1 des entiers signés suivants

Partie A — Coder ces nombres en complément à 1 (5 bits)

Pour un négatif : on écrit d'abord sa valeur absolue, puis on inverse tous les bits.

- $(9)_{10}$

9 en binaire (positif) = 01001

→ $(9)_{10} = 01001$

- $(-9)_{10}$

$|-9| = 9 = 01001$

inversion des bits = 10110

→ $(-9)_{10} = 10110$

- $(6)_{10}$

6 en binaire (positif) = 00110

→ $(6)_{10} = 00110$

- $(-6)_{10}$

$|-6| = 6 = 00110$

inversion des bits = 11001

→ $(-6)_{10} = 11001$

- $(-1)_{10}$

$|-1| = 1 = 00001$

inversion des bits = 11110

→ $(-1)_{10} = 11110$

- $(-15)_{10}$

$|-15| = 15 = 01111$

inversion des bits = 10000

→ $(-15)_{10} = 10000$

- Les deux zéros

+0 = 00000

-0 = 11111 (inversion de 00000)

La présence de deux écritures pour zéro est LE défaut du complément à 1.

Partie B — Décoder ces mots (complément à 1, 5 bits)

Règle : si le bit de gauche est 0 → nombre positif, lu directement.

Si le bit de gauche est 1 → nombre négatif : on inverse les bits pour obtenir la valeur absolue.

- 00101

bit de gauche = 0 → positif → 00101 = 5

→ 00101 = $(5)_{10}$

- 11010

bit de gauche = 1 → négatif

inversion : 11010 → 00101 = 5

donc valeur = -5

→ 11010 = $(-5)_{10}$

• **10000**

bit de gauche = 1 → négatif
inversion : 10000 → 01111 = 15
donc valeur = -15
→ **10000 = (-15)₁₀**

• **11111**

bit de gauche = 1 → négatif
inversion : 11111 → 00000 = 0
donc valeur = -0
→ **11111 = (0)₁₀**

• **01111**

bit de gauche = 0 → positif → 01111 = 15
→ **01111 = (15)₁₀**

Partie C — Additions avec « report circulaire »

En complément à 1, on additionne normalement.

RÈGLE SPÉCIALE : si une retenue SORT du bit de gauche (6e bit), on la réinjecte en l'ajoutant au bit de droite du résultat (c'est le report circulaire). S'il n'y a pas de retenue sortante, le résultat est direct.

• **(5) + (-3)**

```
  0 0 1 0 1    (+5)
+ 1 1 1 0 0    (-3)
-----
1 0 0 0 0 1    <- retenue sortante = 1
Report circulaire : on ajoute la retenue au résultat :
  0 0 0 0 1
+           1
-----
  0 0 0 1 0
→ 00010 = (2)10
```

Retenue sortante → report circulaire → 00010 = +2. Vérif : 5 + (-3) = 2.

• **(-5) + (2)**

```
  1 1 0 1 0    (-5)
+ 0 0 0 1 0    (+2)
-----
  1 1 1 0 0    (pas de retenue sortante → aucune correction)
→ 11100 = (-3)10
```

Aucune retenue sortante : résultat direct 11100 = -3. Vérif : -5 + 2 = -3.

• **(7) + (-4)**

```
  0 0 1 1 1    (+7)
+ 1 1 0 1 1    (-4)
-----
1 0 0 0 1 0    <- retenue sortante = 1
Report circulaire : on ajoute la retenue au résultat :
  0 0 0 1 0
+           1
-----
  0 0 0 1 1
→ 00011 = (3)10
```

Retenue sortante → report circulaire → $00011 = +3$. Vérif : $7 + (-4) = 3$.

• **(6) + (-6)**

0	0	1	1	0	(+6)
+	1	1	0	0	(-6)

1	1	1	1	1	(pas de retenue sortante → aucune correction)

→ **11111** = **(0)**₁₀

Cas remarquable : la somme d'un nombre et de son opposé donne 11111, c'est-à-dire -0 (et non +0). Illustration directe du problème des deux zéros.

Partie D — À retenir

- Négatif en complément à 1 = inversion de tous les bits (rien de plus).
- Sur n bits : plage de $-(2^{(n-1)} - 1)$ à $+(2^{(n-1)} - 1)$. Sur 5 bits : de -15 à +15.
- Deux zéros : $+0 = 00000$ et $-0 = 11111$.
- Addition : report circulaire obligatoire quand une retenue sort à gauche.
- Ces défauts (double zéro + correction de retenue) expliquent pourquoi les ordinateurs ont finalement adopté le complément à 2, où le zéro est unique et l'addition ne demande aucune correction.

Exercice 5 : donne le codage en complément à 2 des entiers signés suivants

1. Nombres demandés

- $(9)_{10}$

$9 \geq 0 \rightarrow$ écriture binaire directe sur 8 bits = 00001001

$\rightarrow (9)_{10} = \mathbf{00001001}$

Positif : simple binaire. $9 = 8 + 1$.

- $(-13)_{10}$

$|-13| = 13 = 00001101$

complément à 1 (inversion) = 11110010

puis + 1 = 11110011

$\rightarrow (-13)_{10} = \mathbf{11110011}$

Vérification : 11110011 décode bien en -13.

- $(-127)_{10}$

$|-127| = 127 = 01111111$

complément à 1 (inversion) = 10000000

puis + 1 = 10000001

$\rightarrow (-127)_{10} = \mathbf{10000001}$

$127 = 01111111 \rightarrow$ inversion $10000000 \rightarrow +1 = 10000001$.

2. Cas complémentaires pertinents

- $(0)_{10}$

$0 \geq 0 \rightarrow$ écriture binaire directe sur 8 bits = 00000000

$\rightarrow (0)_{10} = \mathbf{00000000}$

Zéro UNIQUE en complément à 2 (contrairement au complément à 1 qui a +0 et -0).

- $(-1)_{10}$

$|-1| = 1 = 00000001$

complément à 1 (inversion) = 11111110

puis + 1 = 11111111

$\rightarrow (-1)_{10} = \mathbf{11111111}$

Le cas le plus emblématique : -1 s'écrit avec QUE des 1 (11111111).

- $(127)_{10}$

$127 \geq 0 \rightarrow$ écriture binaire directe sur 8 bits = 01111111

$\rightarrow (127)_{10} = \mathbf{01111111}$

Plus grand positif sur 8 bits : $01111111 = 2^7 - 1$.

- $(-128)_{10}$

$|-128| = 128$ dépasse le maximum positif (+127) : cas limite.

On applique quand même la méthode : $128 = 10000000$

complément à 1 (inversion) = 01111111

puis + 1 = 10000000

$\rightarrow -128$ est le seul nombre dont le complément à 2 se retrouve identique à lui-même.

$\rightarrow (-128)_{10} = \mathbf{10000000}$

Plus petit négatif sur 8 bits. La plage est ASYMÉTRIQUE : -128 existe mais +128 n'est pas représentable sur 8

bits signés.

- **$(100)_{10}$**

$100 \geq 0 \rightarrow$ écriture binaire directe sur 8 bits = 01100100

$\rightarrow (100)_{10} = \mathbf{01100100}$

- **$(-100)_{10}$**

$|-100| = 100 = 01100100$

complément à 1 (inversion) = 10011011

puis + 1 = 10011100

$\rightarrow (-100)_{10} = \mathbf{10011100}$

Comparer $100 = 01100100$ et $-100 = 10011100$: le bit de gauche (signe) distingue les deux.

3. À retenir

- Négatif = inversion des bits (complément à 1) PUIS + 1.
- Le bit de gauche joue le rôle de bit de signe : 0 = positif, 1 = négatif.
- Sur n bits : plage de $-2^{(n-1)}$ à $+(2^{(n-1)} - 1)$. Sur 8 bits : -128 à +127 (asymétrique).
- Un SEUL zéro (00000000) : c'est l'avantage décisif sur le complément à 1.

Astuce de contrôle : un nombre + son opposé doit donner 0 sur 8 bits (la retenue qui sort du 8e bit est simplement ignorée). Ex. 13 (00001101) + (-13) (11110011) = 1 00000000 $\rightarrow$ 00000000.

Exercice 6 : donne la représentation décimale des entiers signés suivants (codés en binaire complément à 2)

1. Nombres demandés

- $(1100\ 1101)_2$

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0011 0010

+ 1 : 51

donc valeur = -51

→ $(1100\ 1101)_2 = (-51)_{10}$ [sur 8 bits]

Contrôle : 11001101 vaut 205 en non signé ; $205 - 256 = -51$ (même résultat).

- $(0000\ 1101)_2$

Bit de gauche = 0 → nombre POSITIF, lu directement.

$0000\ 1101 = 13$

→ $(0000\ 1101)_2 = (13)_{10}$ [sur 8 bits]

Positif : $8 + 4 + 1 = 13$.

- $(1001\ 0001\ 0011\ 0010)_2$

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0110 1110 1100 1101

+ 1 : 28366

donc valeur = -28366

→ $(1001\ 0001\ 0011\ 0010)_2 = (-28366)_{10}$ [sur 16 bits]

Sur 16 bits : inversion puis +1 donne 28366, d'où -28366.

- $(0000\ 0000\ 0111\ 0101)_2$

Bit de gauche = 0 → nombre POSITIF, lu directement.

$0000\ 0000\ 0111\ 0101 = 117$

→ $(0000\ 0000\ 0111\ 0101)_2 = (117)_{10}$ [sur 16 bits]

Positif : $64 + 32 + 16 + 4 + 1 = 117$.

2. Cas complémentaires pertinents

- $(1111\ 1111)_2$

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0000 0000

+ 1 : 1

donc valeur = -1

→ $(1111\ 1111)_2 = (-1)_{10}$ [sur 8 bits]

Le motif « tout à 1 » vaut toujours -1 en complément à 2.

- $(1000\ 0000)_2$

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0111 1111

+ 1 : 128

donc valeur = -128

→ $(1000\ 0000)_2 = (-128)_{10}$ [sur 8 bits]

Cas limite : plus petit négatif sur 8 bits = -128 (la plage est asymétrique, +128 n'existe pas).

• **$(0111\ 1111)_2$**

Bit de gauche = 0 → nombre POSITIF, lu directement.

$0111\ 1111 = 127$

→ **$(0111\ 1111)_2 = (127)_{10}$ [sur 8 bits]**

Plus grand positif sur 8 bits = +127.

• **$(1111\ 1111\ 1111\ 1111)_2$**

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0000 0000 0000 0000

+ 1 : 1

donc valeur = -1

→ **$(1111\ 1111\ 1111\ 1111)_2 = (-1)_{10}$ [sur 16 bits]**

-1 s'écrit « tout à 1 » quelle que soit la largeur (ici 16 bits).

• **$(1000\ 0000\ 0000\ 0000)_2$**

Bit de gauche = 1 → nombre NÉGATIF.

inversion des bits : 0111 1111 1111 1111

+ 1 : 32768

donc valeur = -32768

→ **$(1000\ 0000\ 0000\ 0000)_2 = (-32768)_{10}$ [sur 16 bits]**

Plus petit négatif sur 16 bits = -32768.

3. À retenir

- Bit de gauche = bit de signe : 0 → positif (lecture directe), 1 → négatif (inverser puis + 1).
- Méthode alternative pour un négatif : valeur = (lecture non signée) - 2^n . Ex. $11001101 = 205$; $205 - 256 = -51$.
- « Tout à 1 » = -1 ; « 1 suivi de zéros » = plus petit négatif ($-2^{(n-1)}$).
- Plage sur n bits : de $-2^{(n-1)}$ à $(2^{(n-1)} - 1)$. 8 bits : -128..+127 ; 16 bits : -32768..+32767.

Exercice 7 : convertis ces nombres binaires en base 10

• (1000001,111)₂

Bit : 1 0 0 0 0 0 1 , 1 1 1
 Poids : 64 32 16 8 4 2 1 0,5 0,25 0,125

Somme : $64 + 1 + 0,5 + 0,25 + 0,125 = (65,875)_{10}$

Entière 1000001 = $64 + 1 = 65$; fractionnaire 111 = $0,5 + 0,25 + 0,125 = 0,875$. Total 65,875.

• (1111000,10101)₂

Bit : 1 1 1 1 0 0 0 , 1 0 1 0 1
 Poids : 64 32 16 8 4 2 1 0,5 0,25 0,125 0,0625 0,03125

Somme : $64 + 32 + 16 + 8 + 0,5 + 0,125 + 0,03125 = (120,65625)_{10}$

Entière 1111000 = $64 + 32 + 16 + 8 = 120$; fractionnaire 10101 = $0,5 + 0,125 + 0,03125 = 0,65625$. Total 120,65625.

• (0,1)₂

Bit : 0 , 1
 Poids : 1 0,5

Somme : $0,5 = (0,5)_{10}$

Le tout premier bit fractionnaire : $2^{-1} = 0,5$.

• (0,001)₂

Bit : 0 , 0 0 1
 Poids : 1 0,5 0,25 0,125

Somme : $0,125 = (0,125)_{10}$

Un 1 en 3e position fractionnaire : $2^{-3} = 0,125$.

• (11,011)₂

Bit : 1 1 , 0 1 1
 Poids : 2 1 0,5 0,25 0,125

Somme : $2 + 1 + 0,25 + 0,125 = (3,375)_{10}$

Entière 11 = 3 ; fractionnaire 011 = $0,25 + 0,125 = 0,375$. Total 3,375.

• (0,1111)₂

Bit : 0 , 1 1 1 1
 Poids : 1 0,5 0,25 0,125 0,0625

Somme : $0,5 + 0,25 + 0,125 + 0,0625 = (0,9375)_{10}$

$0,5 + 0,25 + 0,125 + 0,0625 = 0,9375$. Plus on ajoute de 1, plus on s'approche de 1.

• (1010,0101)₂

Bit : 1 0 1 0 , 0 1 0 1
 Poids : 8 4 2 1 0,5 0,25 0,125 0,0625

Somme : $8 + 2 + 0,25 + 0,0625 = (10,3125)_{10}$

Entière 1010 = 10 ; fractionnaire 0101 = $0,25 + 0,0625 = 0,3125$. Total 10,3125.

À retenir

- Les poids fractionnaires se DIVISENT par 2 à chaque rang : 0,5 ; 0,25 ; 0,125 ; 0,0625 ; 0,03125...
- (0,111...1)₂ (n uns) vaut $1 - 2^{-n}$: cela tend vers 1 sans jamais l'atteindre (analogie de 0,999... en décimal).
- Contrôle rapide : la partie fractionnaire d'un binaire fini est toujours un multiple de 2^{-k} (k = nombre de

bits après la virgule), donc son écriture décimale est finie.

Exercice 8 : convertis ces nombres réels en base 2

• $(4,5)_{10}$

Partie entière : divisions par 2

$$4 \div 2 = 2 \quad \text{reste } 0$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

→ restes lus de bas en haut = 100

Partie fractionnaire : multiplications par 2

$$0,5 \times 2 = 1 \rightarrow 1 \quad (\text{reste } 0)$$

→ partie fractionnaire = 1

Résultat : $(4,5)_{10} = (100,1)_2$

Cas fini : $0,5 = 0,1$ en binaire. Donc $4,5 = 100,1$.

• $(3,14)_{10}$

Partie entière : divisions par 2

$$3 \div 2 = 1 \quad \text{reste } 1$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

→ restes lus de bas en haut = 11

Partie fractionnaire : multiplications par 2

$$0,14 \times 2 = 0,28 \rightarrow 0 \quad (\text{reste } 0,28)$$

$$0,28 \times 2 = 0,56 \rightarrow 0 \quad (\text{reste } 0,56)$$

$$0,56 \times 2 = 1,12 \rightarrow 1 \quad (\text{reste } 0,12)$$

$$0,12 \times 2 = 0,24 \rightarrow 0 \quad (\text{reste } 0,24)$$

$$0,24 \times 2 = 0,48 \rightarrow 0 \quad (\text{reste } 0,48)$$

$$0,48 \times 2 = 0,96 \rightarrow 0 \quad (\text{reste } 0,96)$$

$$0,96 \times 2 = 1,92 \rightarrow 1 \quad (\text{reste } 0,92)$$

$$0,92 \times 2 = 1,84 \rightarrow 1 \quad (\text{reste } 0,84)$$

$$0,84 \times 2 = 1,68 \rightarrow 1 \quad (\text{reste } 0,68)$$

$$0,68 \times 2 = 1,36 \rightarrow 1 \quad (\text{reste } 0,36)$$

$$0,36 \times 2 = 0,72 \rightarrow 0 \quad (\text{reste } 0,72)$$

$$0,72 \times 2 = 1,44 \rightarrow 1 \quad (\text{reste } 0,44)$$

... le reste ne devient jamais 0 → développement INFINI (on tronque à 12 bits)

→ partie fractionnaire ≈ 001000111101

Résultat : $(3,14)_{10} \approx (11,001000111101)_2$

Développement INFINI : $3,14$ ne se code pas exactement en binaire. C'est exactement le genre de nombre qui provoque une petite erreur d'arrondi en IEEE 754.

• $(0,5)_{10}$

Partie entière : divisions par 2

$$0 \div 2 = 0 \quad \text{reste } 0 \rightarrow \text{partie entière} = 0$$

Partie fractionnaire : multiplications par 2

$$0,5 \times 2 = 1 \rightarrow 1 \quad (\text{reste } 0)$$

→ partie fractionnaire = 1

Résultat : $(0,5)_{10} = (0,1)_2$

Le plus simple : $0,5 = 2^{-1} = 0,1$.

• $(2,25)_{10}$

Partie entière : divisions par 2

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

→ restes lus de bas en haut = 10

Partie fractionnaire : multiplications par 2

$$0,25 \times 2 = 0,5 \rightarrow 0 \quad (\text{reste } 0,5)$$

$$0,5 \times 2 = 1 \rightarrow 1 \quad (\text{reste } 0)$$

→ partie fractionnaire = 01

Résultat : $(2.25)_{10} = (10,01)_2$

$$0,25 = 2^{-2} \rightarrow 0,01. \text{ Donc } 10,01.$$

• $(6,75)_{10}$

Partie entière : divisions par 2

$$6 \div 2 = 3 \quad \text{reste } 0$$

$$3 \div 2 = 1 \quad \text{reste } 1$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

→ restes lus de bas en haut = 110

Partie fractionnaire : multiplications par 2

$$0,75 \times 2 = 1,5 \rightarrow 1 \quad (\text{reste } 0,5)$$

$$0,5 \times 2 = 1 \rightarrow 1 \quad (\text{reste } 0)$$

→ partie fractionnaire = 11

Résultat : $(6.75)_{10} = (110,11)_2$

$$0,75 = 0,5 + 0,25 \rightarrow 0,11. \text{ Donc } 110,11.$$

• $(10,625)_{10}$

Partie entière : divisions par 2

$$10 \div 2 = 5 \quad \text{reste } 0$$

$$5 \div 2 = 2 \quad \text{reste } 1$$

$$2 \div 2 = 1 \quad \text{reste } 0$$

$$1 \div 2 = 0 \quad \text{reste } 1$$

→ restes lus de bas en haut = 1010

Partie fractionnaire : multiplications par 2

$$0,625 \times 2 = 1,25 \rightarrow 1 \quad (\text{reste } 0,25)$$

$$0,25 \times 2 = 0,5 \rightarrow 0 \quad (\text{reste } 0,5)$$

$$0,5 \times 2 = 1 \rightarrow 1 \quad (\text{reste } 0)$$

→ partie fractionnaire = 101

Résultat : $(10.625)_{10} = (1010,101)_2$

$$0,625 = 0,5 + 0,125 \rightarrow 0,101. \text{ Donc } 1010,101.$$

• $(0,1)_{10}$

Partie entière : divisions par 2

$$0 \div 2 = 0 \quad \text{reste } 0 \rightarrow \text{partie entière} = 0$$

Partie fractionnaire : multiplications par 2

$$0,1 \times 2 = 0,2 \rightarrow 0 \quad (\text{reste } 0,2)$$

$$0,2 \times 2 = 0,4 \rightarrow 0 \quad (\text{reste } 0,4)$$

$$0,4 \times 2 = 0,8 \rightarrow 0 \quad (\text{reste } 0,8)$$

$$0,8 \times 2 = 1,6 \rightarrow 1 \quad (\text{reste } 0,6)$$

$$0,6 \times 2 = 1,2 \rightarrow 1 \quad (\text{reste } 0,2)$$

$$0,2 \times 2 = 0,4 \rightarrow 0 \quad (\text{reste } 0,4)$$

$$0,4 \times 2 = 0,8 \rightarrow 0 \quad (\text{reste } 0,8)$$

$$0,8 \times 2 = 1,6 \rightarrow 1 \quad (\text{reste } 0,6)$$

$$0,6 \times 2 = 1,2 \rightarrow 1 \quad (\text{reste } 0,2)$$

$$0,2 \times 2 = 0,4 \rightarrow 0 \quad (\text{reste } 0,4)$$

$$0,4 \times 2 = 0,8 \rightarrow 0 \quad (\text{reste } 0,8)$$

$$0,8 \times 2 = 1,6 \rightarrow 1 \quad (\text{reste } 0,6)$$

... le reste ne devient jamais 0 → développement INFINI (on tronque à 12 b)

its)

→ partie fractionnaire ≈ 000110011001

Résultat : $(0.1)_{10} \approx (0,000110011001)_2$

Le cas d'école : 0,1 donne 0,00011001100110011...(motif 0011 répété), INFINI. C'est la raison pour laquelle 0,1 + 0,2 $\neq$ 0,3 exactement sur un ordinateur.

À retenir

- Une fraction décimale se code EXACTEMENT en binaire seulement si elle s'écrit comme une somme de $1/2, 1/4, 1/8, 1/16 \dots$ (puissances de 2). Ex. 0,5 ; 0,25 ; 0,75 ; 0,625 : finis.
- Sinon (0,1 ; 0,2 ; 3,14 ; 0,3...), le développement est INFINI et périodique : l'ordinateur doit tronquer, d'où les erreurs d'arrondi de la norme IEEE 754.
- Contrôle : on peut reconvertir en base 10 (somme des poids) pour vérifier les cas finis.

Exercice 9 : les codes suivants ont une taille de 16bits, ils ne sont pas signés et donnés en hexadécimal. Calculez leur valeur en binaire et en décimal.

• **(5C8)₁₆**

Chaque chiffre hexa -> 4 bits :

0=0000 5=0101 C=1100 8=1000

Binaire (16 bits) : **0000 0101 1100 1000**

Décimal : $5 \times 256 + 12 \times 16 + 8 \times 1 = \mathbf{(1480)_{10}}$

Sur 16 bits on complète à gauche : 05C8.

• **(849D)₁₆**

Chaque chiffre hexa -> 4 bits :

8=1000 4=0100 9=1001 D=1101

Binaire (16 bits) : **1000 0100 1001 1101**

Décimal : $8 \times 4096 + 4 \times 256 + 9 \times 16 + 13 \times 1 = \mathbf{(33949)_{10}}$

Commence par 8 (bit de gauche = 1) : en NON signé c'est un grand positif.

• **(8052)₁₆**

Chaque chiffre hexa -> 4 bits :

8=1000 0=0000 5=0101 2=0010

Binaire (16 bits) : **1000 0000 0101 0010**

Décimal : $8 \times 4096 + 5 \times 16 + 2 \times 1 = \mathbf{(32850)_{10}}$

Le chiffre 0 donne un quartet nul 0000.

• **(7A0F)₁₆**

Chaque chiffre hexa -> 4 bits :

7=0111 A=1010 0=0000 F=1111

Binaire (16 bits) : **0111 1010 0000 1111**

Décimal : $7 \times 4096 + 10 \times 256 + 15 \times 1 = \mathbf{(31247)_{10}}$

F = 1111 (quartet plein) ; A = 1010.

• **(0000)₁₆**

Chaque chiffre hexa -> 4 bits :

0=0000 0=0000 0=0000 0=0000

Binaire (16 bits) : **0000 0000 0000 0000**

Décimal : 0 = **(0)₁₀**

Valeur minimale sur 16 bits : 0.

• **(FFFF)₁₆**

Chaque chiffre hexa -> 4 bits :

F=1111 F=1111 F=1111 F=1111

Binaire (16 bits) : **1111 1111 1111 1111**

Décimal : $15 \times 4096 + 15 \times 256 + 15 \times 16 + 15 \times 1 = \mathbf{(65535)_{10}}$

Valeur MAXIMALE sur 16 bits : $65535 = 2^{16} - 1$ (tous les bits à 1).

• **(8000)₁₆**

Chaque chiffre hexa -> 4 bits :

8=1000 0=0000 0=0000 0=0000

Binaire (16 bits) : **1000 0000 0000 0000**

Décimal : $8 \times 4096 = \mathbf{(32768)_{10}}$

Seul le bit de poids fort est à 1 : $2^{15} = 32768$. Frontière signé/non signé : en complément à 2 ce même code vaudrait -32768 .

• **(00FF)₁₆**

Chaque chiffre hexa -> 4 bits :

0=0000 0=0000 F=1111 F=1111

Binaire (16 bits) : **0000 0000 1111 1111**

Décimal : $15 \times 16 + 15 \times 1 = (255)_{10}$

Octet de poids faible plein, octet de poids fort nul : 255.

• **(CAFE)₁₆**

Chaque chiffre hexa -> 4 bits :

C=1100 A=1010 F=1111 E=1110

Binaire (16 bits) : **1100 1010 1111 1110**

Décimal : $12 \times 4096 + 10 \times 256 + 15 \times 16 + 14 \times 1 = (51966)_{10}$

Exemple « mnémonique » : C=1100, A=1010, F=1111, E=1110 → 51966.

À retenir

- 1 chiffre hexa = 4 bits ; 16 bits = 4 chiffres hexa = 2 octets. C'est pourquoi l'hexadécimal est si pratique en informatique : il « résume » le binaire sans perte.
- Non signé sur 16 bits : plage 0 à 65535. Un premier chiffre ≥ 8 met le bit de poids fort à 1 (ce serait négatif en complément à 2, mais reste positif ici).
- Contrôle : la valeur décimale doit toujours rester entre 0 et 65535.

Exercice 10 : converti chaque nombre binaire en hexadécimal

• $(1010\ 1100\ 1111)_2$

On regroupe par quartets de 4 bits (depuis la droite) :

1010 1100 1111

Chaque quartet $\rightarrow$ 1 chiffre hexa :

1010=A 1100=C 1111=F

$\rightarrow (1010\ 1100\ 1111)_2 = (ACF)_{16}$ (soit 2767 en décimal)

12 bits = 3 quartets pile : A, C, F $\rightarrow$ ACF.

• $(1111\ 0000\ 1010\ 1010)_2$

On regroupe par quartets de 4 bits (depuis la droite) :

1111 0000 1010 1010

Chaque quartet $\rightarrow$ 1 chiffre hexa :

1111=F 0000=0 1010=A 1010=A

$\rightarrow (1111\ 0000\ 1010\ 1010)_2 = (F0AA)_{16}$ (soit 61610 en décimal)

16 bits = 4 quartets : F, 0, A, A $\rightarrow$ F0AA.

• $(10111)_2$

On complète à gauche par 3 zéro(s) pour avoir un nombre de bits multiple de 4 :

10111 $\rightarrow$ 0001 0111

Chaque quartet $\rightarrow$ 1 chiffre hexa :

0001=1 0111=7

$\rightarrow (10111)_2 = (17)_{16}$ (soit 23 en décimal)

5 bits : NON multiple de 4 $\rightarrow$ on ajoute 3 zéros à gauche (00010111). Étape à ne pas oublier.

• $(110010101)_2$

On complète à gauche par 3 zéro(s) pour avoir un nombre de bits multiple de 4 :

110010101 $\rightarrow$ 0001 1001 0101

Chaque quartet $\rightarrow$ 1 chiffre hexa :

0001=1 1001=9 0101=5

$\rightarrow (110010101)_2 = (195)_{16}$ (soit 405 en décimal)

9 bits $\rightarrow$ on ajoute 3 zéros à gauche pour former 3 quartets (0001 1001 0101 = 195).

• $(11111111)_2$

On regroupe par quartets de 4 bits (depuis la droite) :

1111 1111

Chaque quartet $\rightarrow$ 1 chiffre hexa :

1111=F 1111=F

$\rightarrow (11111111)_2 = (FF)_{16}$ (soit 255 en décimal)

Un octet plein = FF. À retenir : 8 bits = 2 chiffres hexa.

• $(1101\ 1110\ 1010\ 1101)_2$

On regroupe par quartets de 4 bits (depuis la droite) :

1101 1110 1010 1101

Chaque quartet $\rightarrow$ 1 chiffre hexa :

1101=D 1110=E 1010=A 1101=D

$\rightarrow (1101\ 1110\ 1010\ 1101)_2 = (DEAD)_{16}$ (soit 57005 en décimal)

Exemple mnémorique : donne DEAD.

- $(1011\ 1110\ 1110\ 1111)_2$

On regroupe par quartets de 4 bits (depuis la droite) :

1011 1110 1110 1111

Chaque quartet -> 1 chiffre hexa :

1011=B 1110=E 1110=E 1111=F

→ $(1011\ 1110\ 1110\ 1111)_2 = (\text{BEEF})_{16}$ (soit 48879 en décimal)

Autre mnémonique : donne BEEF.

3. À retenir

- 4 bits = 1 chiffre hexa ; 8 bits (1 octet) = 2 chiffres hexa ; 16 bits = 4 chiffres hexa.
- Toujours regrouper depuis la DROITE, et compléter à GAUCHE par des zéros si besoin (les zéros de tête ne changent pas la valeur).
- Contrôle : convertir l'hexa obtenu en décimal doit redonner la valeur du binaire de départ.

Exercice 11 : quelques questions de réflexion...

1. Valeurs minimum et maximum des entiers signés (32 et 64 bits)

Pour des entiers signés en complément à 2 sur n bits, la plage va de $-2^{(n-1)}$ à $+2^{(n-1)} - 1$ (un bit sert au signe ; la plage est légèrement asymétrique : une valeur négative de plus que de positives).

Bits	Minimum	Maximum
32	$-2^{31} = -2\,147\,483\,648$	$+2^{31} - 1 = +2\,147\,483\,647$
64	$-2^{63} = -9\,223\,372\,036\,854\,775\,808$	$+2^{63} - 1 = +9\,223\,372\,036\,854\,775\,807$

Soit environ $\pm 2,1$ milliards sur 32 bits, et $\pm 9,2 \times 10^{18}$ sur 64 bits.

2. Ces nombres signés tiennent-ils sur un seul octet (8 bits) ?

Sur un octet signé (complément à 2), la plage est -128 à $+127$. On répond oui si le nombre est dans cet intervalle.

Nombre	Réponse	Justification
$(128)_{10}$	NON	$128 > 127$: dépasse le maximum positif (+127).
$(-128)_{10}$	OUI	C'est exactement le minimum : 10000000.
$(255)_{10}$	NON	$255 > 127$ (255 tiendrait en NON signé, pas en signé).
$(127)_{10}$	OUI	C'est exactement le maximum : 01111111.

3. Pourquoi « bit de signe + valeur absolue » est une mauvaise idée

Dans ce système, le bit de gauche indique le signe et les bits restants la valeur absolue.

Deux défauts majeurs :

a) Le problème des DEUX zéros.

$$+0 = 0\ 0000000$$

$$-0 = 1\ 0000000$$

Il existe deux écritures pour zéro, ce qui gaspille une combinaison et complique tout test « est-ce égal à 0 ? ».

b) L'arithmétique ne fonctionne pas

Si on additionne bêtement les motifs de bits, le résultat est faux. Exemple, $(+5) + (-5)$ sur 8 bits :

$$+5 = 0\ 0000101$$

$$-5 = 1\ 0000101$$

$$\text{somme brute des bits} = 1\ 0001010 = -10 \quad (\text{FAUX ! on attendait } 0)$$

Il faut donc une logique spéciale : comparer les signes, comparer les valeurs absolues, décider d'additionner ou de soustraire... Le matériel est bien plus compliqué.

En complément à 2, ces deux problèmes disparaissent : un seul zéro, et un simple additionneur suffit $((+5) + (-5))$ donne 0 automatiquement, la retenue sortante étant ignorée). C'est pourquoi c'est le complément à 2 qui est utilisé partout.

4. Le message « Option Info » en binaire (ASCII)

Chaque caractère ASCII occupe 1 octet (8 bits). On lit son code dans la table ASCII puis on le convertit en binaire.

O	ASCII 79	->	01001111
p	ASCII 112	->	01110000
t	ASCII 116	->	01110100
i	ASCII 105	->	01101001
o	ASCII 111	->	01101111
n	ASCII 110	->	01101110
(espace)	ASCII 32	->	00100000
I	ASCII 73	->	01001001
n	ASCII 110	->	01101110
f	ASCII 102	->	01100110
o	ASCII 111	->	01101111

Message complet en binaire (11 octets, séparés par des espaces) :

01001111 01110000 01110100 01101001 01101111 01101110 00100000 01001001
01101110 01100110 01101111

Le message « Option Info » compte 11 caractères, donc **11 octets (88 bits)**.

L'espace est un caractère à part entière (code 32) et compte pour un octet.

5. Le code de 2 octets (AE75)16 : signé et non signé

Étape 1 — conversion en binaire (chaque chiffre hexa = 4 bits) :

A=1010 E=1110 7=0111 5=0101
(AE75)16 = 1010 1110 0111 0101

Étape 2 — le bit de gauche vaut 1.

• NON signé : on lit directement. $10 \times 4096 + 14 \times 256 + 7 \times 16 + 5 = 44661$

• Signé (complément à 2) : le bit de gauche = 1 → nombre NÉGATIF.

inversion : 0101 0001 1000 1010

+ 1 : 0101 0001 1000 1011 = 20875

donc la valeur signée est **-20875**

Contrôle : valeur signée = valeur non signée - 2^{16} = $44661 - 65536 = -20875$. Cohérent.